

# DealerDB — Documentación Técnica de Base de Datos (SQL Server)

| Motor                         | Microsoft SQL Server 2016 o superior (recomendado 2019 / 2022) |
|-------------------------------|----------------------------------------------------------------|
| Herramienta de administración | SQL Server Management Studio (SSMS) 19 o superior              |
| Lenguaje                      | T-SQL                                                          |
| Esquema                       | dbo                                                            |
| Tablas                        | Marca, Modelos, Cliente, Vehiculos, Cotizaciones               |

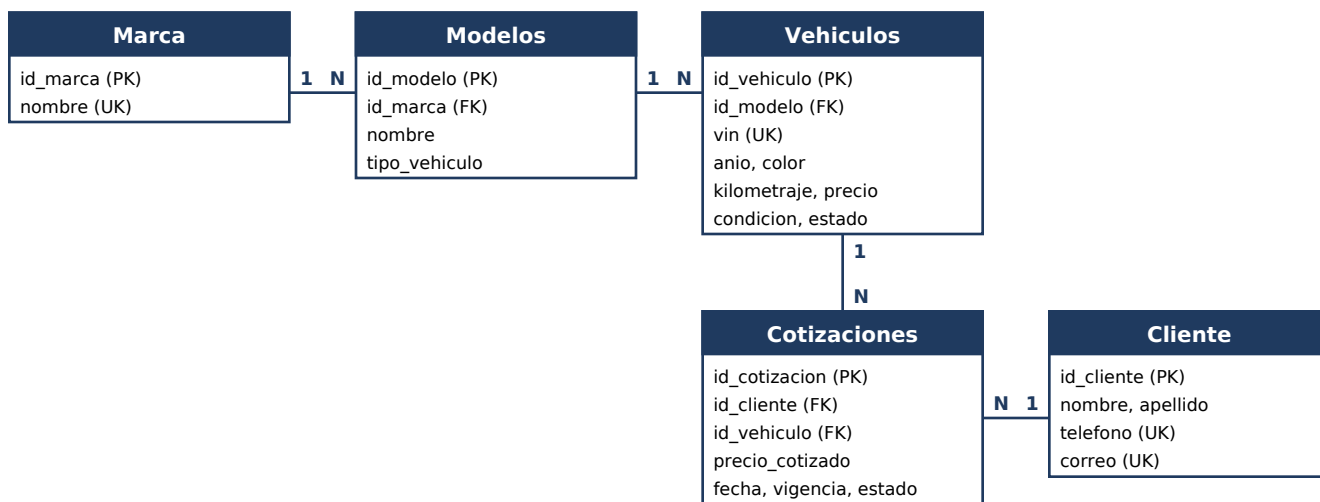
## 1. Propósito y alcance

DealerDB administra la información básica de un concesionario de vehículos:

- Catálogo de **marcas** y **modelos**.
- Control de **unidades físicas** en inventario (identificadas por VIN).
- Registro de **clientes**.
- Generación de **cotizaciones** asociadas a una unidad específica, con integridad referencial completa.

**Fuera de alcance (versión actual):** ventas/facturación, pagos, financiamiento, vendedores y servicio posventa.

## 2. Modelo entidad-relación



**Cardinalidades:** una marca tiene muchos modelos; un modelo tiene muchos vehículos; un cliente puede tener muchas cotizaciones; un vehículo puede aparecer en muchas cotizaciones.

## 3. Diccionario de datos

Los campos de texto usan NVARCHAR (Unicode) para almacenar sin problemas acentos, la ñ y otros caracteres. El VIN usa CHAR(17) porque su longitud es fija y solo contiene caracteres ASCII.

### 3.1 dbo.Marca

| Campo    | Tipo         | Nulo | Reglas            |
|----------|--------------|------|-------------------|
| id_marca | INT          | No   | PK, IDENTITY(1,1) |
| nombre   | NVARCHAR(50) | No   | UNIQUE            |

### 3.2 dbo.Modelos

| Campo         | Tipo         | Nulo | Reglas                                                        |
|---------------|--------------|------|---------------------------------------------------------------|
| id_modelo     | INT          | No   | PK, IDENTITY(1,1)                                             |
| id_marca      | INT          | No   | FK → Marca.id_marca                                           |
| nombre        | NVARCHAR(60) | No   | UNIQUE junto con id_marca                                     |
| tipo_vehiculo | NVARCHAR(20) | No   | CHECK: Sedán, SUV, Hatchback, Pickup, Coupé, Convertible, Van |

### 3.3 dbo.Cliente

| Campo      | Tipo          | Nulo | Reglas            |
|------------|---------------|------|-------------------|
| id_cliente | INT           | No   | PK, IDENTITY(1,1) |
| nombre     | NVARCHAR(60)  | No   |                   |
| apellido   | NVARCHAR(60)  | No   |                   |
| telefono   | NVARCHAR(20)  | No   | UNIQUE            |
| correo     | NVARCHAR(100) | No   | UNIQUE            |

### 3.4 dbo.Vehiculos

| Campo       | Tipo          | Nulo | Reglas                                                    |
|-------------|---------------|------|-----------------------------------------------------------|
| id_vehiculo | INT           | No   | PK, IDENTITY(1,1)                                         |
| id_modelo   | INT           | No   | FK → Modelos.id_modelo                                    |
| vin         | CHAR(17)      | No   | UNIQUE (identificador vehicular)                          |
| anio        | SMALLINT      | No   | CHECK (anio BETWEEN 1900 AND 2030)                        |
| color       | NVARCHAR(30)  | No   |                                                           |
| kilometraje | INT           | No   | DEFAULT 0, CHECK (>= 0)                                   |
| precio      | DECIMAL(12,2) | No   | CHECK (> 0)                                               |
| condicion   | NVARCHAR(10)  | No   | CHECK: Nuevo, Usado                                       |
| estado      | NVARCHAR(15)  | No   | DEFAULT Disponible, CHECK: Disponible, Reservado, Vendido |

### 3.5 dbo.Cotizaciones

| Campo            | Tipo          | Nulo | Reglas                                                            |
|------------------|---------------|------|-------------------------------------------------------------------|
| id_cotizacion    | INT           | No   | PK, IDENTITY(1,1)                                                 |
| id_cliente       | INT           | No   | FK → Cliente.id_cliente                                           |
| id_vehiculo      | INT           | No   | FK → Vehiculos.id_vehiculo                                        |
| precio_cotizado  | DECIMAL(12,2) | No   | CHECK (> 0)                                                       |
| fecha_cotizacion | DATE          | No   | DEFAULT CAST(GETDATE() AS DATE)                                   |
| vigencia_hasta   | DATE          | No   | CHECK (>= fecha_cotizacion)                                       |
| estado           | NVARCHAR(15)  | No   | DEFAULT Pendiente, CHECK: Pendiente, Aceptada, Rechazada, Vencida |

**Nota:** los valores permitidos de estado y tipo\_vehiculo, y las longitudes de las columnas de Marca, Modelos y Cliente, son propuestas razonables. Ajústalos a los de tu script real.

## 4. Script de creación (T-SQL)

Ejecutar en SSMS. Cada bloque GO se envía como un lote independiente.

```
CREATE DATABASE DealerDB;
GO
USE DealerDB;
GO

CREATE TABLE dbo.Marca (
    id_marca INT IDENTITY(1,1) NOT NULL,
    nombre NVARCHAR(50) NOT NULL,
    CONSTRAINT PK_Marca PRIMARY KEY (id_marca),
    CONSTRAINT UQ_Marca_nombre UNIQUE (nombre)
);
GO

CREATE TABLE dbo.Modelos (
    id_modelo INT IDENTITY(1,1) NOT NULL,
    id_marca INT NOT NULL,
    nombre NVARCHAR(60) NOT NULL,
    tipo_vehiculo NVARCHAR(20) NOT NULL,
    CONSTRAINT PK_Modelos PRIMARY KEY (id_modelo),
    CONSTRAINT UQ_Modelos_marca_nombre UNIQUE (id_marca, nombre),
    CONSTRAINT CK_Modelos_tipo CHECK (tipo_vehiculo IN
        (N'Sedán', N'SUV', N'Hatchback', N'Pickup', N'Coupé', N'Convertible', N'Van')),
    CONSTRAINT FK_Modelos_Marca FOREIGN KEY (id_marca)
        REFERENCES dbo.Marca (id_marca)
        ON UPDATE CASCADE ON DELETE NO ACTION
);
GO

CREATE TABLE dbo.Cliente (
    id_cliente INT IDENTITY(1,1) NOT NULL,
    nombre NVARCHAR(60) NOT NULL,
    apellido NVARCHAR(60) NOT NULL,
    telefono NVARCHAR(20) NOT NULL,
    correo NVARCHAR(100) NOT NULL,
    CONSTRAINT PK_Cliente PRIMARY KEY (id_cliente),
    CONSTRAINT UQ_Cliente_telefono UNIQUE (telefono),
    CONSTRAINT UQ_Cliente_correo UNIQUE (correo)
);
GO

CREATE TABLE dbo.Vehiculos (
    id_vehiculo INT IDENTITY(1,1) NOT NULL,
    id_modelo INT NOT NULL,
```

```

vin          CHAR(17) NOT NULL,
anio         SMALLINT NOT NULL,
color        NVARCHAR(30) NOT NULL,
kilometraje  INT NOT NULL CONSTRAINT DF_Vehiculos_km DEFAULT 0,
precio       DECIMAL(12,2) NOT NULL,
condicion    NVARCHAR(10) NOT NULL,
estado       NVARCHAR(15) NOT NULL CONSTRAINT DF_Vehiculos_estado DEFAULT N'Disponible',
CONSTRAINT PK_Vehiculos PRIMARY KEY (id_vehiculo),
CONSTRAINT UQ_Vehiculos_vin UNIQUE (vin),
CONSTRAINT CK_Vehiculos_anio CHECK (anio BETWEEN 1900 AND 2030),
CONSTRAINT CK_Vehiculos_km CHECK (kilometraje >= 0),
CONSTRAINT CK_Vehiculos_precio CHECK (precio > 0),
CONSTRAINT CK_Vehiculos_condicion CHECK (condicion IN (N'Nuevo', N'Usado')),
CONSTRAINT CK_Vehiculos_estado CHECK (estado IN (N'Disponible', N'Reservado', N'Vendido')),
CONSTRAINT FK_Vehiculos_Modelos FOREIGN KEY (id_modelo)
REFERENCES dbo.Modelos (id_modelo)
ON UPDATE CASCADE ON DELETE NO ACTION
);
GO

CREATE TABLE dbo.Cotizaciones (
    id_cotizacion INT IDENTITY(1,1) NOT NULL,
    id_cliente INT NOT NULL,
    id_vehiculo INT NOT NULL,
    precio_cotizado DECIMAL(12,2) NOT NULL,
    fecha_cotizacion DATE NOT NULL
    CONSTRAINT DF_Cotizaciones_fecha DEFAULT (CAST(GETDATE() AS DATE)),
    vigencia_hasta DATE NOT NULL,
    estado NVARCHAR(15) NOT NULL
    CONSTRAINT DF_Cotizaciones_estado DEFAULT N'Pendiente',
    CONSTRAINT PK_Cotizaciones PRIMARY KEY (id_cotizacion),
    CONSTRAINT CK_Cotizaciones_precio CHECK (precio_cotizado > 0),
    CONSTRAINT CK_Cotizaciones_vigencia CHECK (vigencia_hasta >= fecha_cotizacion),
    CONSTRAINT CK_Cotizaciones_estado CHECK (estado IN
        (N'Pendiente', N'Aceptada', N'Rechazada', N'Vencida')),
    CONSTRAINT FK_Cotizaciones_Cliente FOREIGN KEY (id_cliente)
    REFERENCES dbo.Cliente (id_cliente)
    ON UPDATE CASCADE ON DELETE NO ACTION,
    CONSTRAINT FK_Cotizaciones_Vehiculos FOREIGN KEY (id_vehiculo)
    REFERENCES dbo.Vehiculos (id_vehiculo)
    ON UPDATE CASCADE ON DELETE NO ACTION
);
GO

```

## Datos de ejemplo (opcional)

```

INSERT INTO dbo.Marca (nombre) VALUES (N'Toyota'), (N'Honda');

INSERT INTO dbo.Modelos (id_marca, nombre, tipo_vehiculo) VALUES
(1, N'Corolla', N'Sedán'),
(2, N'CR-V', N'SUV');

INSERT INTO dbo.Cliente (nombre, apellido, telefono, correo) VALUES
(N'María', N'Pérez', N'809-555-0101', N'maria.perez@ejemplo.com');

INSERT INTO dbo.Vehiculos (id_modelo, vin, anio, color, kilometraje, precio, condicion)
VALUES
(1, '1HGCM82633A004352', 2024, N'Blanco', 0, 1250000.00, N'Nuevo'),
(2, 'JTDKB20U793456789', 2021, N'Gris', 38500, 1850000.00, N'Usado');

INSERT INTO dbo.Cotizaciones (id_cliente, id_vehiculo, precio_cotizado, vigencia_hasta)
VALUES (1, 1, 1200000.00, DATEADD(DAY, 15, CAST(GETDATE() AS DATE)));

```

## 5. Integridad referencial

Las claves foráneas usan `NO ACTION`, que rechaza el borrado o la actualización si existen filas hijas relacionadas. Todas las claves foráneas usan `ON UPDATE CASCADE ON DELETE NO ACTION`.

| Relación                 | Clave foránea            | Efecto al borrar el padre              |
|--------------------------|--------------------------|----------------------------------------|
| Marca → Modelos          | Modelos.id_marca         | Error si la marca tiene modelos        |
| Modelos → Vehiculos      | Vehiculos.id_modelo      | Error si el modelo tiene unidades      |
| Cliente → Cotizaciones   | Cotizaciones.id_cliente  | Error si el cliente tiene cotizaciones |
| Vehiculos → Cotizaciones | Cotizaciones.id_vehiculo | Error si el vehículo fue cotizado      |

Esto protege el historial operativo. Para "retirar" un registro, cambia su estado en lugar de eliminarlo. Como las llaves primarias son IDENTITY (no se pueden actualizar), ON UPDATE CASCADE es una salvaguarda que en la práctica no se activa.

## 6. Reglas de negocio

- El **VIN** debe tener exactamente 17 caracteres y ser único. El estándar excluye las letras I, O y Q; validarlo en la aplicación.
- Un vehículo Nuevo normalmente tiene kilometraje = 0 (no lo fuerza la base de datos).
- Una cotización solo debería crearse para vehículos Disponible o Reservado (validar en la aplicación o con un trigger).
- El precio cotizado puede diferir del precio de lista (Vehiculos.precio) por descuentos negociados.
- Las cotizaciones Pendiente con vigencia vencida pasan a Vencida con este proceso, que puede programarse con un **trabajo de SQL Server Agent** diario:

```
UPDATE dbo.Cotizaciones
SET estado = N'Vencida'
WHERE estado = N'Pendiente'
AND vigencia_hasta < CAST(GETDATE() AS DATE);
```

## 7. Índices recomendados

**SQL Server no crea índices automáticamente sobre las claves foráneas.** Los índices de PK y UNIQUE sí existen (y UQ\_Modelos\_marca\_nombre ya cubre id\_marca). Se recomienda agregar:

```
CREATE NONCLUSTERED INDEX IX_Vehiculos_id_modelo
ON dbo.Vehiculos (id_modelo);

CREATE NONCLUSTERED INDEX IX_Vehiculos_estado_precio
ON dbo.Vehiculos (estado, precio)
INCLUDE (id_modelo, anio, color, kilometraje, condicion);

CREATE NONCLUSTERED INDEX IX_Cotizaciones_id_cliente
ON dbo.Cotizaciones (id_cliente);

CREATE NONCLUSTERED INDEX IX_Cotizaciones_id_vehiculo
ON dbo.Cotizaciones (id_vehiculo);

CREATE NONCLUSTERED INDEX IX_Cotizaciones_estado_vigencia
ON dbo.Cotizaciones (estado, vigencia_hasta);
```

## 8. Consultas operativas frecuentes

### 8.1 Inventario completo con marca y modelo

```
SELECT v.id_vehiculo, ma.nombre AS marca, mo.nombre AS modelo, mo.tipo_vehiculo,
v.anio, v.color, v.kilometraje, v.precio, v.condicion, v.estado
FROM dbo.Vehiculos AS v
INNER JOIN dbo.Modelos AS mo ON v.id_modelo = mo.id_modelo
INNER JOIN dbo.Marca AS ma ON mo.id_marca = ma.id_marca
ORDER BY ma.nombre, mo.nombre, v.anio DESC;
```

## 8.2 Vehículos disponibles ordenados por precio

```
SELECT ma.nombre AS marca, mo.nombre AS modelo, v.anio, v.color,
       v.kilometraje, v.precio, v.condicion
FROM dbo.Vehiculos AS v
INNER JOIN dbo.Modelos AS mo ON v.id_modelo = mo.id_modelo
INNER JOIN dbo.Marca AS ma ON mo.id_marca = ma.id_marca
WHERE v.estado = N'Disponible'
ORDER BY v.precio ASC;
```

## 8.3 Historial de cotizaciones por cliente

```
DECLARE @id_cliente INT = 1;  -- usar NULL para ver todos los clientes

SELECT c.id_cotizacion, CONCAT(cl.nombre, N' ', cl.apellido) AS cliente,
       ma.nombre AS marca, mo.nombre AS modelo, v.anio,
       c.precio_cotizado, c.vigencia_hasta, c.estado AS estado_cotizacion
FROM dbo.Cotizaciones AS c
INNER JOIN dbo.Cliente AS cl ON c.id_cliente = cl.id_cliente
INNER JOIN dbo.Vehiculos AS v ON c.id_vehiculo = v.id_vehiculo
INNER JOIN dbo.Modelos AS mo ON v.id_modelo = mo.id_modelo
INNER JOIN dbo.Marca AS ma ON mo.id_marca = ma.id_marca
WHERE @id_cliente IS NULL OR c.id_cliente = @id_cliente
ORDER BY c.fecha_cotizacion DESC;
```

*La versión original no filtraba por cliente pese al título; se añadió el parámetro @id\_cliente.*

## 8.4 Cotizaciones pendientes que vencen en los próximos 7 días

```
SELECT c.id_cotizacion, CONCAT(cl.nombre, N' ', cl.apellido) AS cliente,
       cl.telefono, c.vigencia_hasta
FROM dbo.Cotizaciones AS c
INNER JOIN dbo.Cliente AS cl ON cl.id_cliente = c.id_cliente
WHERE c.estado = N'Pendiente'
      AND c.vigencia_hasta BETWEEN CAST(GETDATE() AS DATE)
      AND DATEADD(DAY, 7, CAST(GETDATE() AS DATE))
ORDER BY c.vigencia_hasta;
```

## 8.5 Resumen de inventario disponible por marca

```
SELECT ma.nombre AS marca, COUNT(*) AS unidades,
       CAST(AVG(v.precio) AS DECIMAL(12,2)) AS precio_promedio
FROM dbo.Vehiculos AS v
INNER JOIN dbo.Modelos AS mo ON mo.id_modelo = v.id_modelo
INNER JOIN dbo.Marca AS ma ON mo.id_marca = ma.id_marca
WHERE v.estado = N'Disponible'
GROUP BY ma.nombre
ORDER BY unidades DESC;
```

## 8.6 Los 5 vehículos disponibles más económicos

```
SELECT TOP (5) ma.nombre AS marca, mo.nombre AS modelo, v.anio, v.precio
FROM dbo.Vehiculos AS v
INNER JOIN dbo.Modelos AS mo ON v.id_modelo = mo.id_modelo
INNER JOIN dbo.Marca AS ma ON mo.id_marca = ma.id_marca
WHERE v.estado = N'Disponible'
ORDER BY v.precio ASC;
```

# 9. Mantenimiento y buenas prácticas

### Respaldo completo con verificación:

```
BACKUP DATABASE DealerDB
TO DISK = N'C:\Respaldos\DealerDB_Full.bak'
WITH COMPRESSION, CHECKSUM, INIT, STATS = 10;
```

### Usuario de aplicación con permisos mínimos:

```
CREATE LOGIN dealer_app WITH PASSWORD = N'<contraseña-segura>;
```

```
USE DealerDB;
CREATE USER dealer_app FOR LOGIN dealer_app;
ALTER ROLE db_datareader ADD MEMBER dealer_app;
ALTER ROLE db_datawriter ADD MEMBER dealer_app;
```

- **Modelo de recuperación:** usar FULL en producción con respaldos periódicos del log de transacciones; SIMPLE es suficiente en desarrollo.
- **Año máximo:** el límite 2030 está fijo en el CHECK; revisarlo periódicamente.
- **Datos personales:** telefono y correo son datos personales; restringir permisos de lectura y considerar Always Encrypted o enmascaramiento dinámico de datos.
- **Fechas:** GETDATE() usa la hora del servidor; si hay usuarios en varias zonas horarias, evaluar SYSUTCDATETIME().

## 10. Historial de versiones

| Versión | Cambios                                                                                                                           |
|---------|-----------------------------------------------------------------------------------------------------------------------------------|
| 1.0     | Documentación original                                                                                                            |
| 1.1     | Diagrama ER corregido, diccionario completo, DDL, reglas de negocio, índices y consultas adicionales                              |
| 2.0     | Adaptación completa a Microsoft SQL Server (T-SQL): tipos de datos, IDENTITY, NO ACTION, consultas, índices, respaldo y seguridad |